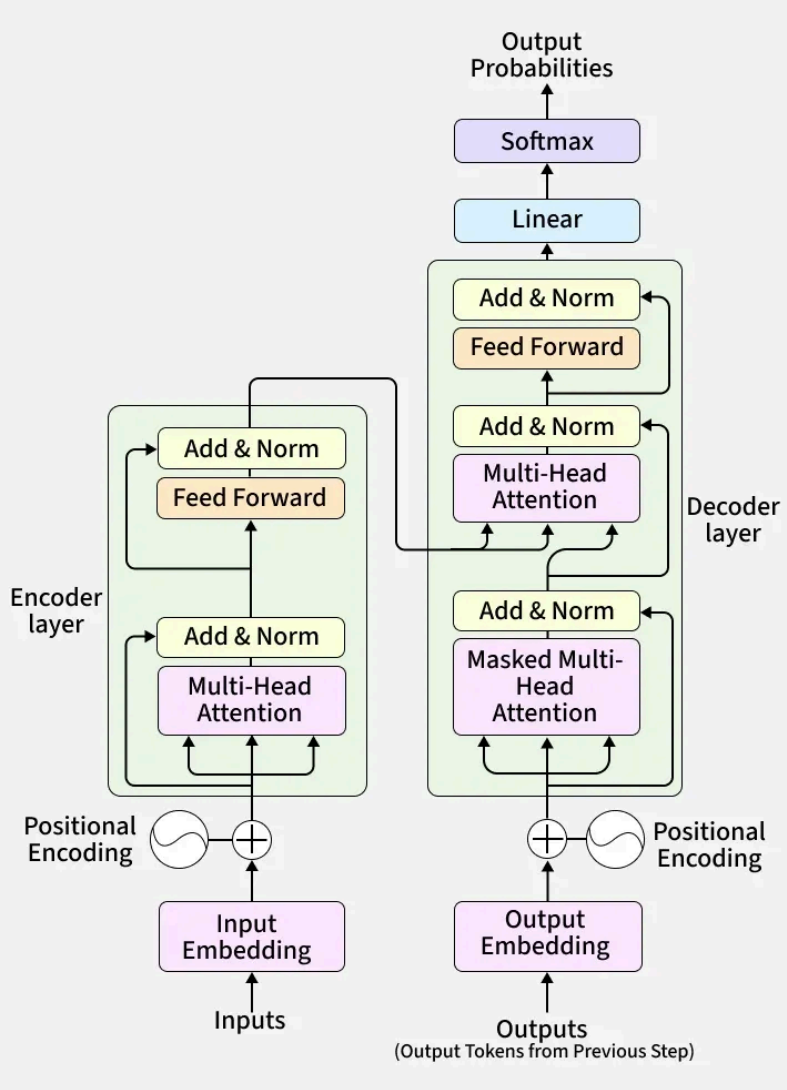
Transformer Architecture

Transformer is a neural network architecture used for various machine learning tasks, especially in natural language processing and computer vision. It focuses on understanding relationships within data to process information more effectively.

- Uses attention mechanisms to capture relationships between inputs
- Processes entire sequences at once instead of step by step
- Improves performance on tasks involving context and dependencies
- Widely used across NLP, vision and other AI applications



1. Self Attention Mechanism
The self attention mechanism allows transformers to determine which words in a sentence are most relevant to each other. This is done using a scaled dot-product attention approach

Each word in a sequence is mapped to three vectors:

- Query (Q)
- Key (K)
- Value (V)

These scores determine how much attention each word should pay to others.

2. Multi-Head Attention
Instead of one attention mechanism, transformers use multiple attention heads running in parallel. Each head captures different relationships or patterns in the data, enriching the model's understanding.

3. Positional Encoding
Unlike RNNs, transformers lack an inherent understanding of word order since they process data in parallel. To solve this problem Positional Encodings are added to token embeddings providing information about the position of each token within a sequence.

4. Position-wise Feed-Forward Networks
The Feed-Forward Networks consist of two linear transformations with a ReLU activation. It is applied independently to each position in the sequence.This transformation helps refine the encoded representation at each position.

5. Add & Norm (Residual Connections and Layer Normalization)
Each sub-layer in the transformer (such as self-attention and feed-forward networks) is followed by an Add & Norm step.

Residual Connection (Add): The input of a layer is added to its output. This helps in preserving information and prevents vanishing gradients in deep networks.

Layer Normalization (Norm): The combined output is normalized to stabilize training and improve convergence.
This structure ensures better gradient flow and allows training of deep transformer models efficiently.

6. Embeddings

Transformers cannot work with raw words as they need numbers. So, each input token (word or subword) is converted into a vector, called an embedding.

Both encoder input tokens and decoder input tokens are converted into embeddings.
These embeddings are trainable, meaning the model learns the best numeric representation for each token.
The same weight matrix is shared for Encoder embeddings, Decoder embeddings and the final linear layer before softmax
The embeddings are scaled by model to keep values stable before adding positional encoding.

7. Encoder-Decoder Architecture

The encoder-decoder structure is key to transformer models. The encoder processes the input sequence into a vector, while the decoder converts this vector back into a sequence. Each encoder and decoder layer includes self-attention and feed-forward layers.

Pretraining vs Fine-Tuning

Pre-training:
The primary goal of pre-training is to expose a Transformer model to a massive dataset of unlabeled text, allowing it to learn the fundamental patterns and structures inherent in language. This vast data often comes from the internet, encompassing books, articles, websites, and more. By processing this diverse information, the model develops a broad understanding of grammar, semantics, and common-sense knowledge.

Several key tasks are employed during pre-training to facilitate this learning:

Masked Language Modeling (MLM): In this technique, a certain percentage of the tokens (words or subwords) in the input text are randomly masked. The model's objective is then to predict these masked tokens based on the surrounding context. This forces the model to understand the relationships between words and capture contextual meaning. For example, in the sentence "The cat sat on the [MASK]," the model would learn to predict "mat."

- Next Sentence Prediction (NSP) : While its effectiveness is debated in more recent research, NSP was an early pre-training task. The model was presented with pairs of sentences and had to predict whether the second sentence was the subsequent sentence in the original document. This aimed to help the model understand the relationships and coherence between sentences.
- Causal Language Modeling (CLM): This task is the core of generative models like GPT. The model is trained to predict the next token in a sequence, given all the preceding tokens. By repeatedly predicting the next word, the model learns the statistical patterns of language generation. For instance, given "Once upon a," the model learns to predict likely next words like "time" or "day."

The benefits of pre-training are substantial. The model learns rich and generalizable language representations, capturing both semantic (meaning) and syntactic (structure) information. This pre-trained knowledge provides a strong

foundation, enabling the model to perform well on a variety of downstream tasks even with limited task-specific data.

Fine-tuning:
Once a Transformer model has been pre-trained on a massive dataset, the next crucial step is fine-tuning. The goal of fine-tuning is to adapt the pre-trained model to perform a specific NLP task using a smaller, labeled dataset relevant to that task.

The fine-tuning process typically involves taking the pre-trained model and adding a small task-specific layer on top (e.g., a classification layer). The entire model, or a significant portion of it, is then trained on the labeled task-specific data. Because the model has already learned general language representations during pre-training, it requires significantly less labeled data and training time to achieve high performance on the target task compared to training a model from scratch.

Common fine-tuning tasks and approaches include:

- Text Classification: Adding a classification layer to the encoder and training it to assign a category label to the input text (e.g., sentiment analysis, topic classification).
- Named Entity Recognition (NER): Adding a token classification layer to the encoder, where the model predicts a label (e.g., Person, Organization, Location) for each token in the input sequence.
- Question Answering: Modifying the architecture to predict the start and end spans of the answer within a given context passage based on a question.
- Machine Translation: Fine-tuning a pre-trained encoder-decoder Transformer on parallel text data (sentences in two languages and their corresponding translations).
- Text Generation: Fine-tuning a pre-trained causal language model on task-specific data or providing specific prompts to guide the generation process.

The benefits of fine-tuning are significant. It allows us to achieve high performance on specific tasks with considerably less labeled data, which can be expensive and time-consuming to acquire. Fine-tuning also results in faster training times, as the model starts with a strong prior knowledge of language learned during pre-training.

Tokenization & Embeddings

Tokenization

Part one of the process, Tokenization breaks the text block into small units of text for the models to analyse. These resulting "tokens" can be:

- whole words,
- parts of words (useful for complex or uncommon words),
- single characters (e.g. individual letters or numbers),
- or even punctuation.

The complete set of tokens, outputted by the tokenization process, is known as an LLMs “vocabulary”. This is the range of tokens that the LLM will be able to recognise and generate. At the time of writing, modern LLMs typically have vocabularies ranging from 30,000 to 100,000 tokens.

As with people, the size of an LLMs vocabulary impacts how well it can understand and generate text. An LLM with an extensive vocabulary is able to handle more words and understand greater concepts. However, unlike with people, bigger vocabularies have a distinct drawback for LLMs. Models with larger tokens are more computationally intensive to run, which affects the speed and costs of using the model. As such, the best performing LLMs strike a balance between model efficiency and output quality.

Embeddings

Using tokenization, the block of text is broken down into parts for the LLM to process. However, these units of language must first be converted into numbers before the LLM can understand and work with them. This is where embeddings come into play.

LLMs use embeddings throughout every layer of their neural network to understand the relationships, context, and meanings of words and tokens. An embedding is a numerical representation of these concepts for a given token, defined as a high-dimensional vector, i.e. a very long list of numbers. At the time

of writing, a typical token vector can consist of hundreds to thousands of floating-point numbers.

The embedding vector defines a token’s meaning as a position in a multi-dimensional space. The closer two vectors are in this “embedding” space, the closer their relationship and their meanings are to each other. So while it sounds complicated dealing with thousands of floating point numbers, conceptually it is quite simple; the more similar the numbers, the more similar the words.

For example, "cat" will have more similar embedding values to "lion" than it does to "moon". Embeddings also encode the context of a token, meaning, the token for "hat" in "hi-hat" will have different embedding values than that of "hat" in "bowler hat". This is what enables Vector Search, the process of searching for words that are conceptually similar rather than spelt the same (as in Keyword Search).
.